

Laboratorio 02 · Actualización de MongoDB 7.0 a 8.0

Duración: 25 minutos · Carpeta de trabajo: `dia1/`

Objetivos

Vas a actualizar una instancia de MongoDB 7.0 con datos a la versión 8.0 siguiendo el procedimiento oficial: comprobar la *feature compatibility version* (FCV), sustituir los binarios, verificar y, solo al final, subir la FCV.

Contexto

Una actualización de versión mayor en MongoDB tiene dos fases separadas a propósito. En la primera cambias los binarios, pero la base de datos sigue escribiendo los datos en un formato compatible con la versión anterior: la FCV sigue en 7.0 y todavía puedes volver atrás cambiando solo los binarios. En la segunda subes la FCV a 8.0, se activan las funcionalidades nuevas que cambian el formato en disco y, a partir de ese momento, la vuelta atrás ya no es trivial.

Las actualizaciones deben ser **consecutivas**: de 6.0 a 7.0, de 7.0 a 8.0. No se puede saltar de 6.0 a 8.0 directamente.

Paso 1 · Arrancar la instancia 7.0 y cargar datos

Comprueba que `dia1/.env` contiene `LEGACY_TAG=7.0-ubi9` y arranca:

```
docker compose up -d legacy
docker compose exec toolbox mongosh "mongodb://legacy:27017" --eval "var
N_PEDIDOS=20000" /scripts-comunes/datos.js
```

Paso 2 · Comprobaciones previas

```
docker compose exec toolbox mongosh "mongodb://legacy:27017"
```

```
> db.version()
> db.adminCommand({ getParameter: 1, featureCompatibilityVersion: 1 })
```

```
> db.getSiblingDB("tienda").pedidos.countDocuments()
```

Anota la versión, la FCV y el número de pedidos. Antes de una actualización real harías además una copia de seguridad, revisarías las *release notes* y las incompatibilidades, y comprobarías que tus drivers soportan la versión de destino.

Paso 3 · Sustituir los binarios

Para limpiar el servidor, cambia la imagen y vuelve a arrancar. El volumen de datos es el mismo:

```
docker compose stop legacy
```

Edita `dia1/.env` y cambia la línea a `LEGACY_TAG=8.0-ubi9`. Después:

```
docker compose up -d legacy
docker compose logs legacy | grep -iE "build
info|featureCompatibility|error" | head
```

Paso 4 · Verificar tras el cambio de binarios

```
> db.version()
> db.adminCommand({ getParameter: 1, featureCompatibilityVersion: 1 })
> db.getSiblingDB("tienda").pedidos.countDocuments()
```

1. ¿Qué versión tienes ahora? ¿Y qué FCV?
2. ¿Qué harías si en este punto la aplicación empezara a fallar?

Paso 5 · Subir la FCV

Cuando la aplicación lleva un tiempo razonable funcionando bien con los nuevos binarios:

```
> db.adminCommand({ setFeatureCompatibilityVersion: "8.0", confirm: true
})
> db.adminCommand({ getParameter: 1, featureCompatibilityVersion: 1 })
```

Prueba a ejecutar el comando sin `confirm: true` y lee el mensaje de error.

▼ Ver solución

Tras el paso 3 tienes MongoDB 8.0.x con FCV "7.0". Si la aplicación falla en ese punto, el rollback es sencillo: vuelves a poner `LEGACY_TAG=7.0-ubi9` y reinicias, porque los datos siguen en formato compatible con 7.0. Una vez subida la FCV a 8.0, volver a 7.0 exige

primero bajar la FCV (con las limitaciones que indique la documentación de downgrade) o restaurar un backup. Desde MongoDB 7.0, `setFeatureCompatibilityVersion` exige `confirm: true` precisamente para que nadie lo ejecute sin ser consciente de ello.

Y en un replica set, ¿cómo sería?

Igual, pero **nodo a nodo** (rolling upgrade): primero los secundarios, uno a uno, esperando a que cada uno vuelva a estado `SECONDARY` y sin lag; después `rs.stepDown()` en el primario, se actualiza, y por último se sube la FCV una sola vez desde el nuevo primario. Lo practicaremos como mantenimiento rolling el día 2.

Limpieza

```
docker compose stop legacy
```